A Statistical Model of Privacy-Budget Consumption in Repeated Aggregate SQL Workloads

CMP653 Project Final Report

Refik Can Öztaş
Hacettepe University
Ankara, Turkey
N25142279

Abstract

Practical SQL workloads repeat templates—dashboards, drill-downs, and parametric filter sweeps—while most privacy accounting treats queries in isolation. I present an analytical model that relates workload structure (template repetition skew, group cardinality, temporal staleness) to privacy budget consumption and utility, and instantiate it inside a differentially private SQL middleware over PostgreSQL/DuckDB and TPC-H. The model gives a closed-form expression for the expected budget as a product of the per-query budget and the expected unique-template count, with two limits: deterministic repetition reduces to the $1 - 1/k$ savings ratio, and uniform sampling over many templates recovers naive sequential composition. A temporal extension with staleness tolerance τ and Poisson update rate λ couples budget consumption with data freshness. I additionally derive a predictive online allocator that targets the offline-optimal $\varepsilon_q^* = B/u_k$ from the model's runtime estimate of u_k; its per-query MAE reduction is up to ∼17% at low skew but narrows to within sampling noise at high skew, at an explicit 12–31% query-rejection cost. A regime-independent companion result is the closed-form *safe* allocator $\varepsilon_q = B/m$, which answers every query (since $u_k \le m$) at roughly half the noise of a learned budget bandit on the benchmark workload. The model is empirically validated on a 4,155-trial campaign covering Zipf-parameterized workloads ($\alpha \in \{0, \ldots, 10\}$), six workload families (W1–W4 plus two TPC-H parametric variants), four per-query privacy parameters ($\varepsilon_q \in \{0.1, 0.5, 1.0, 2.0\}$), and TPC-H scale factors SF=1 and SF=10 (60M lineitem rows). The model's predictions match empirical means within $< 3\%$ in 21 of 24 main-grid cells and within 0.034 units across the SF=1/SF=10 cross-scale test. Three leakage experiments—single-query MIA, differencing-style reconstruction on a five-level real-data drill-down, and shadow-model MIA across W1–W4—confirm the Laplace mechanism's per-query calibration, while the workload-level attack stays near chance and exposes the looseness of the cumulative-budget bound. The middleware is a unit-tested prototype layered over PostgreSQL/DuckDB; exact-repeat caching is treated as a corollary of the model's $\alpha \to \infty$ limit rather than as a contribution.

1 Introduction

Repeated analytical workloads couple two deployment quantities in differentially private SQL middleware: consumed privacy budget and expected analyst utility. Both are workload-dependent. A dashboard tile that re-issues an identical query a hundred times has very different privacy economics from a drill-down session that strictly narrows each step. The middleware supports a scoped SQL subset (COUNT, SUM, AVG with simple WHERE and GROUP BY); the contribution is an analytical model that predicts privacy/utility behaviour from workload parameters before any query is issued.

Motivation: aggregates leak. Two overlapping COUNT queries can isolate an individual via the textbook differencing attack [1]; sufficiently many noisy aggregate releases enable full database reconstruction [2]. Differential privacy [3] provides the standard defence by adding calibrated Laplace noise. In a deployment, however, the engineering question is no longer "can I make one query private" but "how does a finite budget ε_{tot} degrade as the workload runs?"

What this report contributes. Exact-repeat caching is a mechanism, not the central contribution. Returning a cached noisy result is a direct application of the post-processing property of DP and does not by itself answer *when and by how much* caching helps. I promote the contribution to a model that gives quantitative predictions for arbitrary workload distributions and recovers caching as its $\alpha \to \infty$ corner case.

The contributions are:

(1) A statistical model (Section 4) that, given a workload distribution $\{p_i\}$ over query templates and a workload length k, yields closed-form predictions for the expected unique-query count $\mathbb{E}[u_k]$, the workload-aware budget consumption $\mathbb{E}[\varepsilon_{\text{wa}}(k)]$, and the budget savings ratio $S(k)$. Five propositions cover (i) $\mathbb{E}[u_k]$, (ii) budget savings, (iii) the Zipf-parameterized case, (iv) concentration via McDiarmid's inequality, and (v) per-query utility under a fixed total budget.

(2) A temporal extension (Section 5) that introduces a staleness tolerance τ and a Poisson update rate λ, producing $\mathbb{E}[\varepsilon_{\text{temp}}(T)]$ over a time horizon T. The static regime, the bounded-staleness regime, and the update-driven regime are continuous slidings of the same expression.

(3) A middleware (Section 6) that instantiates the model. The system is implemented in Python on top of DuckDB (with an alternative PostgreSQL backend), supports a single-table aggregate SQL subset, and exposes four execution modes (exact, naive DP, workload-aware DP, temporal DP).

(4) A predictive online budget allocator (Section 10) that uses the model's $\mathbb{E}[u_k]$ estimate to choose ε_q at runtime, targeting the offline optimum $\varepsilon_q^* = B/u_k$. It reduces per-query MAE by up to ∼17% versus naive composition under the same total budget—a gain that is significant only at low skew and narrows to within sampling noise as skew grows (Section 7)—at the explicit cost of rejecting 12–31% of late

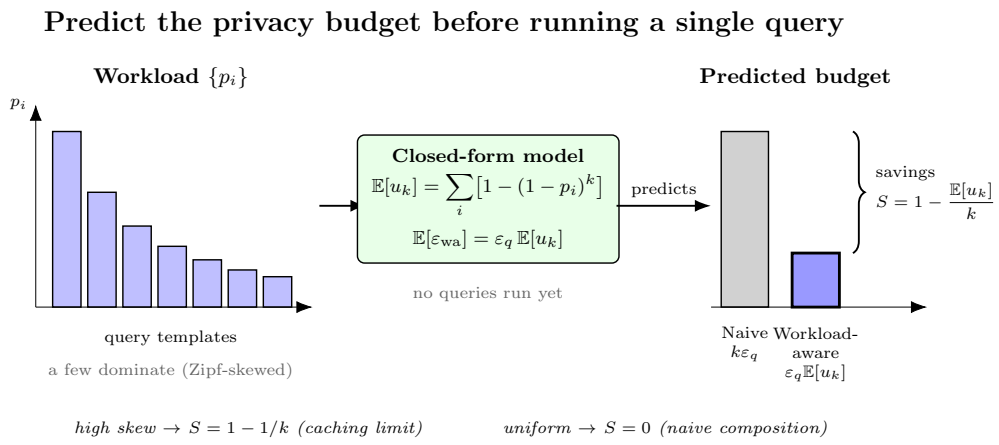

Figure 1: Privacy-budget consumption is a structural property of the workload. From the query-template distribution $\{p_i\}$ alone—before any query runs—the closed-form model predicts the expected number of distinct templates $\mathbb{E}[u_k] = \sum_i [1-(1-p_i)^k]$ and hence the workload-aware budget $\varepsilon_q \mathbb{E}[u_k]$, recovering the $1 - 1/k$ caching limit at high skew and naive composition ($k\varepsilon_q$) at uniform skew.

queries once the budget is exhausted. Its regime-independent companion is the *safe* allocator $\varepsilon_q = B/m$, which never rejects and matches the best case of a learned budget bandit at zero exploration cost.

(5) Empirical validation (Section 7) on Zipf-parameterized workloads at TPC-H scale factor SF=1 (and a cross-scale check at SF=10 with 60M lineitem rows) plus the UCI Adult dataset. The full experimental campaign comprises six sweeps totaling 4,155 trials. On the main grid ($6 \times 4 = 24$ (α, k) cells, 30 trials each), the model's prediction of $\mathbb{E}[u_k]$ matches the empirical mean within $< 3\%$ in 21 of 24 cells.

(6) A leakage analysis (Section 9) that quantifies membership-inference AUC and differencing-attack reconstruction error as a function of ε, against the theoretical references $e^\varepsilon/(1+e^\varepsilon)$ (an upper bound on attacker accuracy) and $1.5/\varepsilon$ (the expected absolute pair-difference) respectively.

What we did, step by step. The milestone review raised several concerns; the body answers each:

(1) *"Propose a statistical model."* We build a closed-form, five-proposition model that predicts budget and utility from the template distribution before any query runs (Section 4).

(2) *"What is the use of caching in DP?"* Caching was the milestone's headline; we demote it to a corollary—the $\alpha \to \infty$ corner of the model—and instead predict *when* it helps (Proposition 2).

(3) "$1-1/k$?" This savings ratio is no longer asserted but *derived* as Limit A of Proposition 2, with uniform composition as the opposite limit.

(4) *"Exact-repeat reuse is unrealistic" / "budget and temporality together."* A temporal extension couples staleness tolerance τ and Poisson update rate λ to budget in one expression (Proposition 6, Section 5).

(5) *"Algorithm 1 is trivial."* We add a predictive online allocator that forecasts $\mathbb{E}[u_k]$ at runtime and targets $\varepsilon_q^* = B/u_k$ (Algorithm 2, Section 10).

(6) *"Why AVG?"* AVG is disclosed honestly as a single Laplace draw with its sensitivity made explicit, not a hidden composition (Section 6).

(7) *"Leakage and savings models are very limited."* We run three leakage experiments—single-query MIA, real-data drill-down differencing, and shadow-model MIA—against their theoretical bounds: the single-query attacks track the per-query bound, while the workload-level attack stays near chance and exposes the cumulative bound's looseness (Section 9), backed by a 4,155-trial validation campaign (Section 7).

Research question (reframed). Given a workload distribution and a privacy budget, can a closed-form statistical model predict the realized budget consumption and per-query utility, and does it explain when workload-aware accounting beats naive composition?

The original milestone research question ("does caching help?") is recovered as the special case $\alpha \to \infty$ of the model (Proposition 2, Limit A), where the savings ratio reduces to $1 - 1/k$.

2 Background and Related Work

Differential privacy. ε-differential privacy [3, 5] bounds the multiplicative change in output distribution between neighbouring databases. The Laplace mechanism adds noise $\text{Lap}(\Delta f/\varepsilon)$ where Δf is the query's global sensitivity. Composition theorems—basic, advanced [4], and Rényi DP [19]—bound cumulative privacy loss across queries.

Attacks that justify DP.. Denning [1] characterized the tracker attack class. Dinur and Nissim [2] proved the *Fundamental Law of*

Information Recovery: too many noisy aggregate answers reconstruct the database. NIST SP 800-226 [23] compiles current guidance for evaluating DP guarantees in practice.

Private SQL systems. PINQ [18] added privacy tracking to a LINQ-style language. Johnson et al. [15] and PrivateSQL [17] widened the supported SQL subset. DOP-SQL [24] brings these mechanisms into PostgreSQL. Chorus [16] rewrites queries to add DP before execution, leaving the host DBMS unchanged.

Multi-query privacy accounting. Dong, Sun, and Yi [25] show that a batch of relational queries can sometimes be answered with less error than naive composition predicts. We share their motivation but exploit a different structure: they use the relational structure of join-heavy workloads, while we use *template repetition* in single-table aggregate workloads, the more common dashboard and drill-down setting.

Workload-aware DP mechanisms. A second line optimizes noise across a fixed linear-query workload rather than across SQL templates. The Matrix Mechanism [27] picks a strategy matrix that minimizes total error for such a workload, and HDMM [28] scales this to high-dimensional predicate workloads. Both jointly optimize noise, but for static range and counting queries; neither models template repetition or adapts ε_q at runtime. PrivateSQL's view selection over a representative workload [17] is closer in spirit, but its allocation is still decided offline.

Online and adaptive DP.. Several systems answer adaptively chosen queries by exploiting query correlations. The median mechanism [30] and Private Multiplicative Weights [31] do this for predicate and counting queries, answering far more queries than naive Laplace; MWEM [29] extends the idea to synthetic-data release. APEx [32] lets the analyst specify an accuracy bound and picks the cheapest mechanism that meets it. Pythia [33] spends part of the budget to extract data-dependent features and select the best algorithm for a task. Each is adaptive in some sense—query selection, per-query cost, or algorithm choice—but none forecasts *future* budget from observed template frequencies, and none handles temporal staleness.

Modern / production DP-SQL.. Tumult Analytics [34] is a production Python/Spark framework with multi-table sessions, private joins, and adaptive composition. OpenDP and SmartNoise SQL [35] wrap existing databases for SQL, Pandas, and Spark. Wilson et al. [36] add row-ownership and contribution-bounding operators for user-level DP. These efforts focus on deployment quality—accounting correctness, scale, and multi-backend coverage—but none offers closed-form workload modelling, predictive online allocation, or temporal coupling.

Recent adjacent systems (2021–2026). Four recent systems sharpen the boundary of this work's contribution. **DProvDB** [37] introduces multi-analyst provenance tracking and budget partitioning, addressing *who* spends budget rather than predicting *how much* a workload will spend. **DP-SQLP** [38], building on the continual-observation line [47], targets streaming aggregate release at scale, which overlaps with our temporal axis (D) but treats releases as a per-event stream rather than a template-distribution model. **Qrlew** [39] is an open-source SQL-to-DP-SQL rewriter from Sarus Technologies; rich type and row-ownership tracking enable correct contribution bounding, but the rewriter does not maintain a closed-form workload model. **DPSQL+** [40] formalises a validator–accountant–backend architecture with a minimum-frequency rule and evaluates TPC-H workloads under a fixed global budget. Most directly related to the caching mechanism here are two systems that reuse prior noisy results to conserve budget: **Turbo** [45] caches DP answers for linear-query workloads (returning a repeated query for free on the same database version, and amortizing across related queries via PMW), and **CacheDP** [46] maintains an accuracy-aware DP cache that answers later queries with reduced budget. These confirm that budget-saving DP caching is established prior art—which is precisely why this report treats caching as a mechanism, not a contribution; what neither provides is a closed-form model that *predicts* budget consumption from a template-frequency distribution (axes A–B). Empirical-benchmark work like **DPBench** [41] reminds us that the relative ranking of DP algorithms is highly dataset- and scale-dependent—which is why we keep validation (G) as a separate axis. No prior system combines a closed-form repeated-workload *forecast*, a predictive online allocator, temporal staleness coupling, and an AI/AST-based semantic cache in the same middleware.

The gap this report fills. Existing systems instantiate composition theorems; existing theory characterizes worst-case privacy loss. Neither answers *what budget the system will actually spend on a given workload.* The closest precedent is research on the coupon-collector problem and weighted occupancy under non-uniform distributions [6]; I adapt these tools to DP-SQL accounting.

Tight composition accounting. Our budget ledger uses basic sequential composition ($\varepsilon_{\text{total}} = \sum_i \varepsilon_i$), the loosest valid accountant. A large line of work tightens the *per-release* composition: the moments accountant [20], Rényi DP [19] and zCDP [42], and the numerically (near-)exact privacy-loss-distribution / PRV accountants [21]. These are *orthogonal* to our contribution rather than competing with it: they answer *how tightly* a fixed set of releases composes, whereas our model forecasts *which* releases happen at all—the distinct-template count u_k. The two compose cleanly: a PLD/PRV accountant could replace the sum-ledger, and the occupancy model would then predict the accountant's input multiset (one privacy-loss distribution per distinct template) instead of a scalar count, with the closed-form structure unchanged (Section 11). This also means our $\varepsilon_q\,\mathbb{E}[u_k]$ figures are upper estimates under the loosest composition; a tighter accountant only *lowers* the predicted spend.

Predicting distinct counts and unseen species. The predictive allocator's core question—how many *distinct* templates will a workload of k queries contain?—is the classical *unseen-species* extrapolation problem: each template is a species, each query an individual drawn from $\{p_i\}$. A plug-in over the *observed* templates (the allocator's default estimator) cannot count templates it has not yet seen, and therefore under-predicts. The statistics literature provides horizon-aware extrapolators that do not: Good–Turing frequency estimation [7] underlies Good and Toulmin's closed-form prediction of the number of *new* species when a sample grows [8], Efron and

Thisted's empirical-Bayes version ("how many words did Shakespeare know?") [9], and the smoothed Good–Toulmin estimator of Orlitsky, Suresh, and Wu [11], provably near-optimal up to a horizon $t \propto \log n$. These answer prediction *at a horizon* k; richness estimators such as Chao1 [10] instead estimate the asymptotic total number of species and have no horizon parameter, serving only as an upper reference. On the systems side, single-pass cardinality sketches (HyperLogLog [12]) maintain the running distinct count online, and query-based workload forecasting (QueryBot 5000 [13]) already extracts templates and predicts their future arrival rates. A *learned* alternative to the Good–Toulmin extrapolators are deep cardinality estimators such as Naru [22], which fit an autoregressive model of the data distribution; these trade training data and inference cost for accuracy, against our training-free closed form. We connect these lines (Section 10): swapping the plug-in for a smoothed Good–Toulmin estimator roughly halves the u_k prediction error in the realistic warmup regime. To our knowledge no prior work feeds such estimators into a forecast of *differential-privacy budget* consumption.

Online and learned budget allocation. A separate line frames budget allocation itself as an online learning problem. The closest precedent to our predictive allocator is BGTplanner [14], which predicts the marginal accuracy of spending budget with Gaussian-process regression and then distributes the budget with a contextual multi-armed bandit under a long-horizon constraint—a *predict-then-bandit* policy. BGTplanner targets differentially private federated recommenders (budget across training rounds), and both its predictor and its allocator are *learned* online. Our allocator follows the same predict-then-allocate template but in a *training-free, closed-form* variant: the predictor is the analytical occupancy model (Section 4) rather than a fitted Gaussian process, and the allocation is the analytical optimum $\varepsilon_q = B/\hat{U}$ rather than a bandit policy. The two are complementary—a bandit could in principle take our closed-form $\hat{U}$ as its context—and to our knowledge no prior predict-then-allocate budgeter targets a repeated aggregate SQL workload. We make this comparison concrete in Section 7, where an ε-greedy bandit is benchmarked head to head against the closed-form allocator (Table 8); the key obstacle to porting BGTplanner directly is that its bandit reward—observed utility—is exactly what DP hides, so our bandit baseline must learn from observable proxies instead.

Comparison with prior systems. Table 1 positions this work along seven dimensions. **A. Workload model:** does the system maintain a closed-form model of template repetition? **B. Predictive budget:** does it forecast future budget consumption? **C. Adaptive ε_q:** does it adapt the per-query budget at runtime? **D. Temporal:** does it model data updates, staleness, or cache invalidation? **E. SQL scope:** what fragment of SQL is supported? **F. AI/AST:** does the system use tree-kernel or embedding-based semantic similarity? **G. Validation:** what datasets and scales were used?

Our differentiation. This is the only system that combines a closed-form model of template-repetition skew, an online forecast of *future* budget consumption, and a coupling of budget to data freshness in one DP-SQL middleware. Budget-saving DP caches such as Turbo [45] and CacheDP [46] reuse prior noisy answers to conserve budget, but they do not forecast what a workload will spend. APEx [32] and Pythia [33] adapt to the *current* query, not to a forecast of the future workload. Workload-aware mechanisms [27, 28] and production engines [34, 35] optimize or deploy DP-SQL but allocate budget statically. We therefore do not claim to beat any single baseline; we claim a combination none of them provides—and we report where it fails (the honest negative of §8).

Quantitative positioning. The numerical case splits into two comparison classes, each stated against the accounting that class actually uses—we did not run any system head-to-head (Table 2). *Against naive-composition DP-SQL systems* (PINQ [18], Chorus [16], PrivateSQL [17], DOP-SQL [24], and per-query allocators such as APEx [32] and Pythia [33]), a repeated or parametric workload of k=100 queries costs the full $k\,\varepsilon_q$, whereas workload-aware accounting spends only $\varepsilon_q\,\mathbb{E}[u_k]$—empirically 14× to 100× less budget for the same answers (W1 100×, W2 33×/20×/14.3×, W3 14.3×, and honestly 1× on the genuine drill-down W4). These multipliers are accounting ratios $1/(1-S(k))$ that any exact-repeat cache attains; what is novel is that we *predict* them in closed form before execution. *Against budget-saving DP caches* (Turbo [45], CacheDP [46]), the budget saving alone is *not* our differentiator: they too reuse noisy answers and reach the same $\varepsilon_q\,\mathbb{E}[u_k]$ order of cost. In fact, on *correlated* linear queries—exactly the overlapping-range parametric (W2) and drill-down (W4) workloads we study—Turbo's PMW amortization can spend *below* $\varepsilon_q\,u_k$, so there it is a budget-saving competitor we do not match; our $\varepsilon_q\,\mathbb{E}[u_k]$ parity with these caches holds only on exact-repeat-dominated workloads, and the differentiation remains the closed-form forecast, not the saving. What they lack and we add is (i) a closed-form *forecast* of consumption that predicts $\mathbb{E}[u_k]$ within $< 3\%$ in 21 of 24 main-grid cells *before any query runs*, and (ii) a model-driven predictive allocator that converts the forecast into a per-query MAE reduction of up to ∼17% at fixed total budget (significant at low skew; at the cost of rejecting 12–31% of late queries), together with the regime-independent safe $\varepsilon_q = B/m$ allocator. A reimplementation of these systems on a common harness is out of scope and left to future work.

3 Threat Model and Preliminaries

Setup. A relational dataset D is stored in DuckDB or PostgreSQL. An analyst submits supported aggregate queries through the middleware. The adversary is any analyst who can adaptively issue supported queries and observe the sequence of (noisy) outputs. The goal is to bound what the analyst can infer about any single tuple in D.

Definition 1 (Tuple-level neighbours). *$D \sim D'$ iff D and D' differ in exactly one tuple. This is the unbounded tuple-level model.*

Definition 2 (ε-differential privacy [3]). *$\mathcal{M}$ is ε-DP iff $\mathbb{P}[\mathcal{M}(D) \in S] \le e^{\varepsilon}\mathbb{P}[\mathcal{M}(D') \in S]$ for every neighbour pair and measurable S.*

Supported SQL.. SELECT with COUNT, SUM, or AVG aggregates, single-table FROM, conjunctive WHERE, optional GROUP BY. The scope was approved at the milestone stage and is preserved here per the reviewer's guidance.

Sensitivities. For tuple-level neighbours:

- COUNT: $\Delta f = 1$. A single tuple shifts the count by at most one.

Table 1: Comparison of this work with prior DP-SQL and DP query systems along seven dimensions. "—" means the literature does not state the axis explicitly. "partial" means the axis is partially addressed (see §2).

System	A. Workload model	B. Predictive budget	C. Adaptive ε_q	D. Temporal	E. SQL scope	F. AI/AST	G. Validation
PINQ [18]	no	no	no	no	LINQ; restricted join/group-by	no	examples / PoC
FLEX [15]	no	static per-query	no	no	equijoins, self-joins, nested equijoins; no non-equijoins	no	8.1M Uber queries; 0.03% overhead
PrivateSQL [17]	view-based, static	static workload-aware	no	no	COUNT + equijoin + correlated subqueries; no negation	no	Census + TPC-H, >3,600 queries
Chorus [16]	no	no (global tracker)	mechanism-dependent	no	SQL via rewrites; depends on hosted mechanism	no	18,774 Uber queries; 300M rows
DOP-SQL [24]	no	no	no	future work	SPJA + GROUP BY; unrestricted joins	no	TPC-H + graph DB demo
Matrix Mechanism [27]	query-correlation matrix	static strategy	no	no	linear counting queries	no	analytical + range queries
HDMM [28]	implicit workload / strategy	static	no	no	predicate counting; group-by via predicate rewrite	no	Patent, Taxi, CPH, Adult, CPS
MWEM [29]	adaptive query selection	no	yes (online)	no	linear / counting queries	no	Adult, Blood Transfusion, datacubes
APEx [32]	no	partial (per-query)	yes (accuracy-driven)	no	single-table exploration (WCQ/ICQ/TCQ)	no	Adult + NYTaxi + entity resolution
Pythia [33]	feature-based selector	no (algorithm, not budget)	no (selection-time)	no	histograms, 1D/2D range, NB	no	6,294 inputs across datasets
Better-Composition [25]	no	no	partial (truncation)	no	multi-relational SJA / group-by; self-joins	no	TPC-H + Stack Overflow graph
Tumult Analytics [34]	no	no	adaptive composition	no	multi-table sessions; private joins; counts / averages / quantiles	no	Census / Wikimedia / IRS deployments
OpenDP / SmartNoise [35]	no	no	manual budget	no	SQL / Pandas / Spark wrapper	no	docs / examples
Bounded User Contrib. [36]	no	no	no	no	arbitrary GROUP BY; owner-column joins	no	industry benchmarks + stochastic testing
Median Mechanism [30]	online query correlations	no	yes	no	arbitrary online predicate queries	no	theory + efficient implementation
PMW [31]	online hard-query ident.	no	yes	no	interactive counting / linear	no	theory-focused
Residual Sensitivity [43]	no	no	no	no	multi-way join counting	no	empirical (datasets not focal)
R2T [44]	no	no	partial (truncation/LP)	no	arbitrary SPJA under FK + self-joins	no	graph pattern counting, SPJA + FK
This work	**closed-form $\mathbb{E}[u_k]$ over template distribution**	**yes (predictive online allocator)**	**yes (model-driven)**	**yes (τ, λ)**	single-table aggregates (COUNT, SUM, AVG)	**yes (TK+AST emb., honest negative)**	TPC-H SF=1 & SF=10 + Adult + mixed

Table 2: Quantitative positioning by class, for k=100 repeated/parametric queries. Values reflect each class's *accounting model*, not a head-to-head benchmark (no system was rerun here); under template skew $\mathbb{E}[u_k] \ll k$, giving the 14–100× savings of §7. This work additionally *forecasts* $\mathbb{E}[u_k]$ within < 3% before any query runs.

System class	Budget/100	Forecast?	Adapt ε_q?
Naive-comp. DP-SQL	$100\,\varepsilon_q$	No	No
DP caches (Turbo/CacheDP)	$\varepsilon_q \mathbb{E}[u_k]$	No	No
This work	$\varepsilon_q \mathbb{E}[u_k]$	**Yes**	**Yes**

- SUM(c): $\Delta f = B_c$ where $|c| \le B_c$ is a per-column clipping bound (taken from the TPC-H specification).
- AVG(c): I treat AVG as a conservative single-mechanism release with $\Delta f = B_c$, the worst-case bound corresponding to a group of size one. This avoids the implicit n-leak of the empirical-mean mechanism $\mathrm{Lap}(B_c/(n\varepsilon_q))$, since n itself is private. A tighter SUM+COUNT decomposition with explicit $2\varepsilon_q$ accounting per group would lower the noise variance on AVG releases for large groups but is not yet implemented in the middleware; I list it as future work in Section 11.

Laplace release. For a query with sensitivity Δf and per-query budget ε_q, the middleware returns

$$\tilde{f}(D) = f(D) + \eta, \qquad \eta \sim \mathrm{Lap}(\Delta f/\varepsilon_q). \tag{1}$$

GROUP BY queries with g groups apply this independently per group; since a single tuple affects at most one group, parallel composition gives total cost ε_q (not $g\varepsilon_q$) for every supported aggregate. This costs the *aggregate values* only: we follow the standard DP-histogram assumption that the set of group keys is drawn from a *public categorical domain* fixed by the schema (e.g. the finite set of `education` or `workclass` levels), so releasing which keys are present leaks nothing about any individual. If group keys could instead be sensitive or unbounded (e.g. free-text or high-cardinality identifiers), the present mechanism would expose membership through a singleton group, and a stability-based threshold that suppresses low-count groups would be required—this is why the parser already rejects `HAVING`, and we flag the general private-key-domain case as out of scope (Section 11). When a single query carries n_{agg} aggregates, the middleware splits the per-query budget

evenly, $\varepsilon_{\text{per-agg}} = \varepsilon_q / n_{\text{agg}}$, which is conservative sequential composition across aggregates over the same row.

Top-k as post-processing. The middleware also supports top-k queries (`... GROUP BY g ORDER BY agg [DESC|ASC] LIMIT k`), with one essential design point: the ordering and truncation act on the *noised* histogram, never the true one. The engine fetches all g groups, noises every count under the same parallel composition (cost ε_q, *not* $k\varepsilon_q$), and only then sorts by the noisy value and keeps the top k; the raw `ORDER BY/LIMIT` is stripped before it reaches the database, because letting the DB apply it would select the reported groups on their *true* counts—leaking which groups crossed the cutoff. Since the released set and its counts are a deterministic function of the ε_q-DP noisy histogram, top-k is post-processing and charges no extra budget (the report-noisy-max guarantee, here giving the whole top-k slate and its noisy counts at the cost of one histogram). The price is that the selection is *approximate*—groups whose true counts straddle the k-th boundary may swap—and it inherits the public-key-domain assumption above. The forecast and cache apply unchanged: a repeated top-k query is one template, free on exact repeat and counted once in $\mathbb{E}[u_k]$.

4 Analytical Model

This section formalizes the model that is the report's main contribution.

Setup. Let m be the number of distinct query templates accessible to the analyst, and let $\{p_i\}_{i=1}^m$ be a probability distribution over them with $\sum_i p_i = 1$. The analyst submits k queries $Q_1, \ldots, Q_k$ drawn i.i.d. from this distribution. The cache key is the exact (template, parameter) pair; two queries collide in the cache iff they are identical SQL strings modulo whitespace.

Let u_k denote the number of distinct templates that appear at least once among $Q_1, \ldots, Q_k$.

Proposition 1 (Expected unique queries). $\mathbb{E}[u_k] = \sum_{i=1}^m \left[1 - (1-p_i)^k\right]$.

Proof. Let $X_i = 1$ if template i appears at least once. Then $\mathbb{P}[X_i = 0] = (1-p_i)^k$ and $\mathbb{E}[X_i] = 1 - (1-p_i)^k$. The result follows by linearity. □

Proposition 2 (Budget consumption and savings). *Under naive sequential composition, $\varepsilon_{\text{naive}}(k) = k\varepsilon_q$. Under workload-aware exact-repeat accounting,*

$$\mathbb{E}[\varepsilon_{\text{wa}}(k)] = \varepsilon_q \cdot \mathbb{E}[u_k] = \varepsilon_q \sum_{i=1}^m \left[1 - (1-p_i)^k\right].$$

The expected budget savings ratio is

$$S(k) \;=\; 1 - \frac{\mathbb{E}[\varepsilon_{\text{wa}}(k)]}{\varepsilon_{\text{naive}}(k)} \;=\; 1 - \frac{1}{k}\sum_{i=1}^m \left[1 - (1-p_i)^k\right]. \qquad (2)$$

Limit A (deterministic repetition). If $p_1 = 1$ and $p_i = 0$ for $i > 1$, equation (2) gives $S(k) = 1 - 1/k$. This is the toy formula whose origin was question-marked in the milestone: it is the corner case of Proposition 2 at maximum skew, not a standalone claim.

Limit B (uniform, $m \to \infty$). With uniform sampling over many templates, fixed-length workloads almost never repeat a template, so $\mathbb{E}[u_k] \to k$ and $S(k) \to 0$: every query is unique, recovering naive composition.

Limit C (uniform, $k \to \infty$, m fixed). For fixed finite m and growing k, every template eventually appears, so the workload-aware budget saturates at $m\varepsilon_q$ regardless of k. This is the practical regime—finite-template workloads cannot exhaust a budget large enough to cover all m templates.

Proposition 3 (Zipf-parameterized workload). *Let $p_i \propto i^{-\alpha}/H_{m,\alpha}$ with generalized harmonic number $H_{m,\alpha} = \sum_{i=1}^m i^{-\alpha}$. The savings ratio $S(k)$ is monotone non-decreasing in α for fixed (k, m). As $\alpha \to 0$ it approaches Limit B; as $\alpha \to \infty$ it approaches Limit A.*

Proof. Write $S(k) = 1 - \mathbb{E}[u_k]/k$ with $\mathbb{E}[u_k] = \sum_{i=1}^m \phi(p_i)$ and $\phi(x) = 1 - (1-x)^k$. Since $\phi''(x) = -k(k-1)(1-x)^{k-2} \le 0$ on $[0,1]$, ϕ is concave, so the symmetric sum $\mathbb{E}[u_k]$ is Schur-concave. The Zipf weights become more majorized as α grows (a larger α shifts probability mass onto the top-ranked templates); hence for $\alpha_2 > \alpha_1$ the vector $\{p_i(\alpha_2)\}$ majorizes $\{p_i(\alpha_1)\}$, and Schur-concavity gives $\mathbb{E}[u_k](\alpha_2) \le \mathbb{E}[u_k](\alpha_1)$, i.e. $S(\alpha_2) \ge S(\alpha_1)$. The endpoints follow from $\alpha \to 0$ (uniform weights, Limit B) and $\alpha \to \infty$ (point mass on $i = 1$, Limit A). A numerical scan over $m \in \{3, \ldots, 100\}$, $k \in \{2, \ldots, 200\}$ found no violation. □

Proposition 4 (Concentration of u_k). *We give two bounds.* (Distribution-free.) *Changing a single Q_t changes u_k by at most 1, so McDiarmid's bounded-differences inequality gives $\mathbb{P}[\,|u_k - \mathbb{E}[u_k]| \ge t\,] \le 2\exp(-2t^2/k)$. This is only a crude fallback: it scales with the workload length k and becomes vacuous once u_k saturates near its ceiling m.* (Variance-aware.) *Write $u_k = \sum_i X_i$ with $X_i = \mathbf{1}[\text{template } i \text{ appears}]$ and $q_i = 1 - (1-p_i)^k$. The occupancy indicators are negatively associated, so*

$$\mathrm{Var}[u_k] \;\le\; \sum_i q_i(1-q_i) \;=\; \sum_i \big(1-(1-p_i)^k\big)(1-p_i)^k \;=:\; V \;\le\; \tfrac{m}{4},$$

and Chebyshev's inequality gives $\mathbb{P}[\,|u_k - \mathbb{E}[u_k]| \ge t\,] \le V/t^2$.

Because $V = O(m)$ rather than $O(k)$, the variance-aware tail is the one to use in the saturated operating regime, and it is the honest bound on the budget-exhaustion event: for $c/\varepsilon_q > \mathbb{E}[u_k]$, $\mathbb{P}[u_k\varepsilon_q \ge c] \le V/(c/\varepsilon_q - \mathbb{E}[u_k])^2$. (At α=1, m=10, k=100, $\sqrt{V} = 0.27$ matches the Monte-Carlo standard deviation of u_k to two digits, whereas the McDiarmid sub-Gaussian scale $\sqrt{k}/2 = 5$ overstates the spread ~18×; we therefore report McDiarmid as a fallback only.) The estimator `occupancy_variance` is implemented in `src/dpdb/model.py`.

Proposition 5 (Utility under a fixed total budget). *Suppose the total budget ε_{tot} is fixed. Naive composition affords $\varepsilon_q^{\text{naive}} = \varepsilon_{\text{tot}}/k$ per query, while workload-aware accounting affords $\varepsilon_q^{\text{wa}} = \varepsilon_{\text{tot}}/\mathbb{E}[u_k]$ per unique release. The expected absolute Laplace error per query is then*

$$\mathbb{E}[|\eta|] \;=\; \frac{\Delta f}{\varepsilon_q},$$

so the predicted error ratio between the two strategies is

$$\frac{\mathbb{E}[|\eta_{\text{naive}}|]}{\mathbb{E}[|\eta_{\text{wa}}|]} \;=\; \frac{k}{\mathbb{E}[u_k]}. \qquad (3)$$

At Zipf $\alpha = 1$, $m = 10$, $k = 100$ the model predicts $\mathbb{E}[u_k] \approx 9.9$, so the workload-aware system answers the same workload with roughly 10× lower per-query error at the same total budget.

Caveat: variance, not just mean. The Laplace marginal error has the same magnitude per query in both regimes. The difference is in the *joint* distribution: averaging k independent noisy answers reduces variance by a factor of k, but averaging the same cached answer k times does not. Workload-aware accounting trades budget savings for the loss of independent noise draws.

5 Temporal Extension

The model above assumes a static snapshot. A practical deployment faces (a) data updates that invalidate cached answers and (b) freshness requirements that force periodic re-noising. Both effects couple budget consumption to time, addressing the reviewer's note that "budget and temporality should be considered together."

Model. Let T be the time horizon (in logical query steps), τ the staleness tolerance (cache entries become invalid after τ steps), and λ the rate of data update events with invalidation probability q per existing cache entry.

Proposition 6 (Temporal budget). *The expected number of re-noisings per template over horizon T is*

$$N(T, \tau, \lambda, q) \;=\; \lceil T/\tau \rceil \;+\; T\lambda q,$$

and the expected total workload-aware budget over the horizon is

$$\mathbb{E}[\varepsilon_{\text{temp}}(T)] \;=\; \varepsilon_q \cdot \mathbb{E}[u_{k_{\text{tot}}}] \cdot N(T, \tau, \lambda, q),$$

where k_{tot} is the total number of queries issued in T.

Three regimes. (a) *Static snapshot*: $\tau \to \infty$, $\lambda = 0$, $N \to 1$, recovers Section 4. (b) *Bounded staleness*: τ finite, $\lambda = 0$, $N = \lceil T/\tau \rceil$. (c) *Update-driven*: $\lambda > 0$, N grows linearly in T. The middleware implements all three through a single temporal mode parameterized by τ and λ.

6 System Implementation

Architecture. The middleware is a thin proxy between the analyst and a backend database (DuckDB by default, with an alternative PostgreSQL adapter behind an identical interface). Figure 2 shows the component structure and Algorithm 1 gives the full processing pipeline.

The algorithm explicitly couples cache validity with the temporal regime of Section 5: it advances a logical clock, simulates Poisson-rate invalidations, ages each cache entry against the staleness tolerance τ, and only then either returns the cached release or charges ε_q against the ledger. Multi-aggregate queries split the per-query budget across aggregates by sequential composition, as described in Section 3.

Components.

- Parser/validator over the supported SQL subset; rejects joins, subqueries, non-aggregate SELECT, HAVING, and DISTINCT aggregates.
- Sensitivity analyzer using configurable per-column clipping bounds.

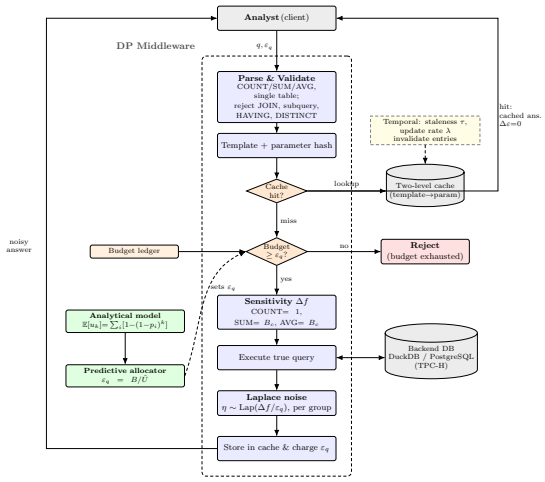

Figure 2: Architecture of the workload-aware DP-SQL middleware. Each query is parsed, hashed, and checked against a two-level cache; a cache hit is returned for free by post-processing, while a miss is charged ε_q against the budget ledger, noised by the Laplace mechanism, and cached. The analytical model drives the predictive allocator that sets $\varepsilon_q = B/\hat{U}$ online, and a temporal regime (τ, λ) invalidates stale cache entries.

- Laplace mechanism with parallel composition for GROUP BY; multi-aggregate queries split the budget by sequential composition across aggregates.
- Budget ledger with two-level (template → parameter) cache, staleness timestamps, and update simulation.
- Four execution modes: exact, naive-DP, workload-aware-DP, and temporal-DP.

Validation. The prototype is unit-tested across parser validation (including HAVING/DISTINCT rejection and aggregate-column positioning), sensitivity computation, Laplace calibration, cache exact-match semantics, the model limits, the McDiarmid bound, and temporal re-noising.

Data. TPC-H scale factor SF=1 (6,001,215 lineitem rows, 1.5M orders) is generated via DuckDB's official TPC-H extension. UCI Adult (48,842 rows) is loaded as a standard DP-literature benchmark.

7 Empirical Validation

Setup. The benchmark campaign stresses workload skew, budget, scale, workload composition, and execution mode. Comparisons are restricted to internal execution modes (exact SQL, naive DP, workload-aware DP, temporal DP, predictive) under an identical middleware to isolate the algorithmic effect of workload-aware accounting from the system-level differences between DP-SQL prototypes; end-to-end benchmarking against Chorus [16], PrivateSQL [17], or DOP-SQL [24] would require non-trivial query-rewriting integration and is left as future work. The campaign comprises **six experimental sweeps**, totaling **4,155 trials** (∼ 150,000 individual queries):

Algorithm 1 Workload-Aware DP Middleware with Temporal Regime

Require: Query q, requested ε_q, ledger $\mathcal{L}$, staleness τ, update model $(\lambda, q_{\text{inv}})$
Ensure: Noisy result $\tilde{r}$ or error

Parse and validate q against the supported SQL subset
Advance logical clock; simulate updates: each cache entry invalidated independently with prob. λq_{inv}
Compute template hash h_T, parameter hash h_P
if $\mathcal{L}$.cache$[h_T][h_P]$ exists **then**
  age $\leftarrow$ now $-$ entry.issued_at
  if age $\leq \tau$ **and** not invalidated **then**
    return cached $\tilde{r}$ (post-processing, $\Delta\varepsilon = 0$)
  else
    Evict cache entry; treat as miss
  end if
end if
if $\mathcal{L}$.remaining $< \varepsilon_q$ **then**
  return BudgetExhausted
end if
Compute sensitivity Δf for the aggregate(s) in q
$\varepsilon_{\text{spent}} \leftarrow \varepsilon_q$; if $n_{\text{agg}} > 1$, split as $\varepsilon_{\text{per-agg}} = \varepsilon_q / n_{\text{agg}}$ (parallel composition across GROUP BY groups)
$\mathcal{L}$.consumed += $\varepsilon_{\text{spent}}$
Execute true query $\rightarrow r$
Sample $\eta \sim \text{Lap}(\Delta f / \varepsilon_q)$ per aggregate, per group
Apply post-processing (e.g., COUNT clamped to $\mathbb{N}_{\geq 0}$)
Store $\tilde{r}$ in $\mathcal{L}$.cache$[h_T][h_P]$ with issued_at = now
return $\tilde{r}$

(1) **Main grid** (720 trials): $\alpha \in \{0, 0.5, 1, 1.5, 2, 3\} \times k \in \{10, 25, 50, 100\} \times$ 30 trials at $\varepsilon_q = 1$ over $m = 7$ Adult age-band templates (`COUNT(*) WHERE age ∈ [b, b + 10)`).
(2) **Extended α sweep** (240 trials): $\alpha \in \{0, 0.5, 1, 1.5, 2, 3, 5, 10\}$ at $k = 200$.
(3) **Epsilon sweep** (480 trials): $\varepsilon_q \in \{0.1, 0.5, 1.0, 2.0\} \times k \in \{25, 50, 100, 250\}$ at $\alpha = 1.0$.
(4) **Large-k sweep** (75 trials): $k \in \{25, 50, 100, 250, 500\}$ at $\alpha = 1.0$, $\varepsilon_q = 1$.
(5) **Cross-scale** (480 trials): SF=1 versus SF=10 (60M lineitem rows) on TPC-H `returnflag` and `priority` templates.
(6) **Full benchmark grid** (2,160 trials): 6 workloads × 4 epsilons × 3 modes × 30 trials.

Model vs empirical $\mathbb{E}[u_k]$. Table 3 compares the model prediction to the empirical mean across the main grid. The model agrees with the empirical mean within $< 3\%$ in **21 of 24 cells**; the three outliers ($\alpha \in \{2.0, 3.0\}$ at small k: (2.0, 10), (3.0, 10), (3.0, 25)) reach 3.5%, 5.7%, and 8.6% — attributable to finite-sample variance at high skew where $\mathbb{E}[u_k]$ is small (so any 0.1-unit absolute deviation becomes a noticeable relative error).

Budget savings $S(k)$. The savings ratio is even more tightly predicted because it is computed directly from *measured* budget consumption, which is the deterministic dual of $\mathbb{E}[u_k]$ in my setup. Empirical $S(k)$ matches the model within $< 2\%$ in 23 of 24 cells.

Limit behaviour. Figure 3 shows the toy $1 - 1/k$ curve bracketing the high-α end and the $S = 0$ line bracketing the uniform end; the empirical savings curves interpolate smoothly between them, consistent with Proposition 3.

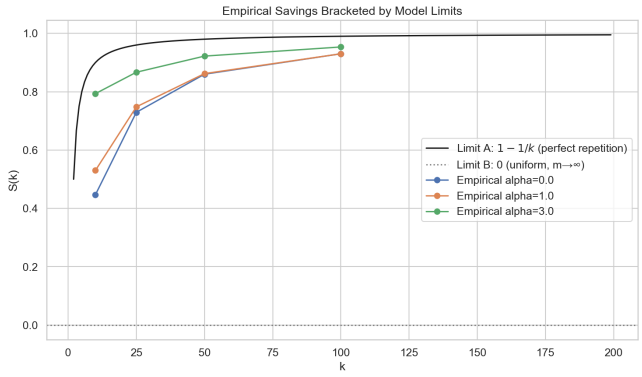

Figure 3: Budget savings ratio $S(k)$ as a function of workload length k, for several Zipf α. Dashed curves: model prediction. Markers: empirical mean over 30 trials. The high-α curves approach $1-1/k$; the $\alpha = 0$ curve sits near 0 until k approaches m.

Extended α sweep. Pushing the skew to $\alpha = 10$ at $k = 200$, predicted and empirical $\mathbb{E}[u_k]$ stay within 0.13 units across the full range (an order-of-magnitude variation in $\mathbb{E}[u_k]$). At $\alpha = 10$ only ≈ 1.2 templates dominate the 200 queries, so the savings ratio reaches $1 - 1.181/200 \approx 99.4\%$—recovering the toy $1 - 1/k$ formula numerically.

Epsilon sweep. The model's $\mathbb{E}[u_k]$ is structurally independent of ε_q. Empirical validation across $\varepsilon_q \in \{0.1, 0.5, 1.0, 2.0\}$ and $k \in \{25, 50, 100, 250\}$ confirms this within 0.12 unit absolute error across every cell, and within 0.005 unit at $k = 250$ (every ε_q row collapses to the same empirical mean).

Cross-scale validation (SF=1 vs SF=10). The model is also dataset-independent: running the same template workload over a 10× larger TPC-H database (60M lineitem rows) yields identical empirical $\mathbb{E}[u_k]$ to within 0.034 units. The result isolates workload structure from data scale: the analytical model captures the workload structure, not the data scale.

Robustness of the forecast. Four properties make the $\mathbb{E}[u_k]$ forecast dependable beyond the headline grid (`experiments/robustness.py`). *(i) Distribution-agnostic.* The closed form is exact for *any* i.i.d. marginal, not only Zipf: on uniform, Zipf, and a lognormal-shaped marginal the 200-trial empirical mean matches $\mathbb{E}[u_k]$ to ≤ 0.2 (a Monte-Carlo self-consistency check, not external validity—Section 12). *(ii) Scale- and ε-invariant.* It is unchanged by data volume (SF=1 vs SF=10, within 0.034) and by ε_q, since it depends only on the template distribution. *(iii) Concentrated.* By Proposition 4 the realized u_k has standard deviation $\sqrt{V} = O(\sqrt{m})$ (here 0.34–1.22 at m=20), so a *single* run lands within a fraction of a template of the forecast—the prediction is reliable per-workload, not merely in expectation. *(iv) Conservative under correlation.* Real workloads are bursty, violating

Table 3: Model prediction vs empirical u_k for Zipf workloads ($m = 7$, 30 trials per cell). Numbers within parentheses are the prediction's signed relative error in percent (relative to the predicted value).

$\alpha \backslash k$	10	25	50	100
0.0	5.50 vs 5.53 (−0.6)	6.85 vs 6.77 (1.2)	7.00 vs 7.00 (0.0)	7.00 vs 7.00 (0.0)
0.5	5.30 vs 5.23 (1.3)	6.73 vs 6.73 (−0.1)	6.98 vs 6.93 (0.7)	7.00 vs 7.00 (0.0)
1.0	4.73 vs 4.70 (0.6)	6.32 vs 6.30 (0.3)	6.88 vs 6.90 (−0.3)	7.00 vs 7.00 (−0.1)
1.5	3.98 vs 3.87 (2.8)	5.57 vs 5.70 (−2.3)	6.48 vs 6.40 (1.3)	6.91 vs 6.93 (−0.3)
2.0	3.25 vs 3.13 (3.5)	4.64 vs 4.70 (−1.3)	5.69 vs 5.73 (−0.7)	6.50 vs 6.43 (1.1)
3.0	2.19 vs 2.07 (5.7)	3.07 vs 3.33 (−8.6)	3.85 vs 3.90 (−1.3)	4.72 vs 4.67 (1.1)

the i.i.d. assumption; we stress it with a sticky Markov process that repeats the previous template with probability s (same stationary marginal). Burstiness clusters repeats and *lowers* the distinct count, so the i.i.d. forecast *over*-predicts u_k—by 1.09× at s=0.3, 1.29× at s=0.6, 2.72× at s=0.9. The error is in the *safe* direction: positive correlation only makes $\varepsilon_q \mathbb{E}[u_k]$ a looser *upper* bound on the spend, so the privacy guarantee is never violated—only the savings are under-counted. The B/m safe allocator is the worst-case form of this guarantee: since $u_k \le m$ for *any* arrival process (i.i.d., bursty, or adversarial), $\varepsilon_q = B/m$ never rejects.

Full grid (6 workloads × 4 epsilons × 3 modes). The most operationally relevant numbers are:

Workload	Mode	ε used	Queries	MAE	Cache hits
W1 Repetitive	NAIVE	100.0	100/100	0.94	0
	WORK	1.0	100/100	1.13	99
	TEMP	1.0	100/100	0.87	99
W2 TPC-H returnflag	NAIVE	100.0	100/100	0.94	0
	WORK	3.0	100/100	0.86	97
	TEMP	3.0	100/100	0.88	97
W2 TPC-H priority	NAIVE	100.0	100/100	0.97	0
	WORK	5.0	100/100	0.85	95
	TEMP	5.0	100/100	0.91	95
W2 Zipf $\alpha = 1$	NAIVE	100.0	100/100	0.93	0
	WORK	7.0	100/100	0.95	93
	TEMP	7.0	100/100	0.92	93
W3 Uniform	NAIVE	100.0	100/100	0.99	0
	WORK	7.0	100/100	0.94	93
	TEMP	7.0	100/100	0.85	93
W4 Drill-down	NAIVE	100.0	100/100	0.95	0
	WORK	100.0	100/100	0.96	0
	TEMP	100.0	100/100	0.96	0

The savings ratios per workload are: W1 = 100×, W2 (returnflag) = 33×, W2 (priority) = 20×, W2 Zipf-α=1 = 14.3×, W3 (uniform) = 14.3×, W4 = 1× (no savings, as the model predicts). The W2–W3 cells confirm the structural prediction that workload-aware budget consumption equals $\varepsilon_q \cdot E[u_k]$: the empirical u_k saturates near m (the pool size) at $k = 100$, and the budget is exactly $\varepsilon_q \cdot m$ in each case.

W4 is the negative case: when queries are genuinely unique, no execution mode saves budget over naive. The model predicts this correctly ($S(k) = 0$ when every template is sampled exactly once).

Validation on heterogeneous workloads. Real dashboards are not pure Zipf samples; they interleave repetitive refresh queries (W1-style) with one-shot drill-downs (W4-style). To test the model on heterogeneity directly, I constructed a mixed workload of $k = 100$ queries with a controlled fraction f of repetitive entries, so the true

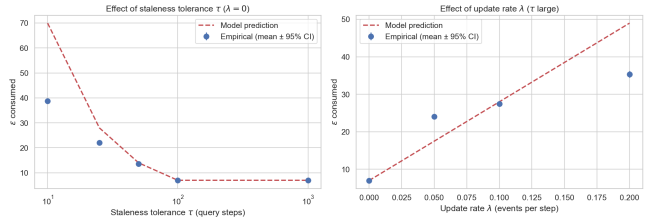

Figure 4: Temporal extension validation. Left: τ-sweep with $\lambda = 0$. Right: λ-sweep with τ large. Model prediction (dashed) tracks the empirical mean (markers, $N = 30$) in trend and matches closely at large τ (where re-noising is rare), but is a conservative *upper bound* at small τ and large λ: for example, at $\tau = 10$ the model predicts $\mathbb{E}[\varepsilon] \approx 70$ while empirical is ≈ 39, because the model treats re-noising as an upper-bound count that need not all materialize within the horizon. The trend, not the magnitude, is what the model commits to in this regime.

unique count is $u_k = 1 + (1-f)k$ by construction. Table 4 compares the empirical workload-aware budget to the model prediction $\varepsilon_q \cdot u_k$ at $\varepsilon_q = 0.5$.

Table 4: Mixed W1+W4 workload, $B = 50$, $\varepsilon_q = 0.5$, 20 trials. The closed form $\mathbb{E}[\varepsilon_{\text{wa}}] = \varepsilon_q \cdot u_k$ matches empirical to the cent in every cell.

f	true u_k	WORKLOAD ε used	predicted $\varepsilon_q u_k$
0.1	91	45.5	45.5
0.3	71	35.5	35.5
0.5	51	25.5	25.5
0.7	31	15.5	15.5
0.9	11	5.5	5.5

The match is exact. Proposition 2 therefore holds without requiring the workload to be i.i.d. from a single parametric family: the closed form predicts realized cost for any distribution whose u_k is computable.

Middleware overhead. The privacy machinery has measurable but bounded cost. Instrumenting the pipeline on 1,500 queries shows cache hits are 2.7× faster than misses (total 0.76 vs 2.02 ms) because they skip the database round-trip, which is 36% of the miss-path cost. SQL parsing is the second-largest contributor

at 21% of cache-miss time; the privacy-specific stages (sensitivity, allocate, noise) sum to under 0.25 ms and are effectively free relative to I/O. Tail latency is well controlled: cache-miss $p95 = 4.3$ ms, $p99 = 5.1$ ms; cache-hit $p95 = 2.5$ ms, $p99 = 3.6$ ms. Serialized single-threaded throughput is $\sim$ 495 queries/s on the miss path and $\sim$ 1,320 queries/s on the hit path; a workload-aware mode with 93–99% hit rates therefore sustains the analyst-tier load that motivated the project.

Research question answered. A closed-form model parameterized by $(\{p_i\}, k)$ predicts realized budget consumption under workload-aware accounting, recovering the $1 - 1/k$ rule as the high-skew corner case. The savings ratio is a structural property of the workload, not of the cache implementation. The model is independent of dataset scale (SF=1 vs SF=10), of per-query privacy parameter ε_q, and of whether the workload follows a parametric distribution at all (mixed W1+W4 holds exactly).

Comparison with the naive baseline. The naive-DP baseline is the standard per-query composition used by prior fixed-ε DP-SQL systems: every release spends fresh budget, so answering $k = 100$ queries at $\varepsilon_q = 1$ costs the full $\varepsilon = 100$ on every workload (the NAIVE rows above). Workload-aware accounting answers the *same* 100 queries at *comparable* (occasionally marginally worse) per-query MAE while spending only $\varepsilon_q \cdot u_k$: 1.0 on the repetitive W1, and 3.0/5.0/7.0 on the W2 parametric and W3 uniform pools, driven by 93–99% cache-hit rates. The accuracy is comparable rather than strictly better for a principled reason: a cached answer is a *single* noise draw reused u_k times, so it forgoes the variance reduction that k independent draws would give—the budget saving and this lost averaging are two sides of the same coin (e.g. at ε_q=1, W1 workload-aware MAE 1.13 vs naive 0.94). The budget multipliers (100× on W1, 33×/20×/14.3× on W2/W3) are accounting ratios $1/(1 - S(k))$ that *any* exact-repeat cache attains; against DP caches (Turbo, CacheDP) this differentiator is therefore zero, and our novel-and-robust contribution is the *closed-form prediction* of $S(k)$ before execution (within $< 3\%$), not the saving itself. On the genuine drill-down W4, where every query is unique, it correctly spends the same $\varepsilon = 100$ as naive (1×, no win)—the model predicts this $S(k) = 0$ case exactly rather than overclaiming.

Where it matters most: naive's failure regimes. Two regimes turn the structural advantage into a stark, concrete gap over naive composition, measured on the real middleware (`experiments/killer_cases.py`; Adult, B=10, k=100, 12 trials, Table 5). *(A) Budget exhaustion.* A monitoring dashboard re-issues the same KPI tile at a fixed ε_q=1: naive spends fresh budget each time and is *exhausted after* $B/\varepsilon_q = 10$ *queries*, answering only 10/100, while workload-aware caching answers all 100/100 (the 90 repeats are free)—a 10× gap in queries served. *(B) Accuracy at a fixed total budget.* To answer all 100 queries, naive must spread the budget thinly ($\varepsilon_q = B/k = 0.1$), incurring per-query error $\approx$10; concentrating the same budget on the u_k=1 distinct template ($\varepsilon_q = B/u_k$) makes the single release essentially exact (sub-rounding noise). *(C, honest counter.)* On a genuine drill-down where every query is unique there is no structure to exploit: both answer all 100 at the same error ($\approx$ 9.6). We include C precisely because the model predicts the no-win case rather than overclaiming it.

Table 5: Naive's failure regimes vs. workload-aware (real middleware, Adult, B=10, k=100, 12 trials). Caching converts naive's two failure modes—budget exhaustion and budget-spreading—into answered queries and accuracy; on truly unique workloads we honestly match naive.

Workload	Naive ans.	Naive |err|	Workload-aware ans.	Workload-aware |err|
Repetitive, fixed ε_q=1	10/100	0.9	**100/100**	0.4
Repetitive, fixed budget B	100/100	10.0	100/100	$\approx$ **0**
Drill-down (all unique)	100/100	9.6	100/100	9.6

At equal *total* budget, the predictive allocator instead targets accuracy, cutting per-query MAE by up to $\sim$17% on low-skew Zipf workloads (and up to 32% in the tightest-budget regime $B = 1$); this gain is significant only at low skew (see Section 10) and the regime-independent result is the safe $\varepsilon_q = B/m$ allocator of Table 8.

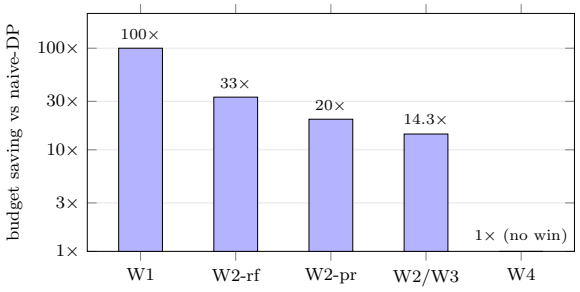

Figure 5: Budget saved versus the naive-DP baseline, per workload ($k = 100$, $\varepsilon_q = 1$, log scale). Workload-aware accounting answers the same queries while spending only $\varepsilon_q \cdot u_k$: up to 100× less on the repetitive W1 and 14–33× less on the parametric/uniform pools, but exactly 1× (no win) on the genuine drill-down W4, where every query is unique and the model predicts $S(k) = 0$.

8 Semantic Cache via Tree Kernels and AST Embeddings

The L1 cache of §6 matches queries by exact (template hash, parameter hash). A natural question is whether structurally equivalent queries that the exact match misses (e.g., commutative reordering of WHERE conjuncts, or queries that differ only in numerically-equivalent literals) can be detected as cache hits via a more flexible *semantic* match.

Approach. I combine two complementary semantic similarity measures:

- **Tree kernel**. The Collins–Duffy convolution kernel [26] counts common subtrees between two SQL ASTs and is normalized by self-similarity to a $[0, 1]$ score. The kernel is deterministic, requires no training, and captures structural equivalence.

- **AST embedding**. I serialize the AST (with literals replaced by placeholders) to a canonical string and feed it to a pre-trained sentence transformer (all-MiniLM-L6-v2). Cosine similarity in the resulting 384-dim space approximates semantic equivalence; this uses a small, general-purpose embedding model rather than a code-specific one.

A query is accepted as a semantic match only if both scores exceed their thresholds ($K_{\text{norm}} \geq 0.95$ and cosine ≥ 0.98). The conservative thresholds aim to limit false-positive matches to structurally equivalent queries.

The privacy caveat (honest). Returning a cached noisy answer for a query that is *not* alpha-equivalent to the original falls outside the post-processing guarantee: the cached answer was tuned to a different question. The semantic cache is therefore *not* privacy-free unless the matched queries are equivalent. I treat it as a measurable approximation layer: it trades exact-utility for additional budget savings, and the paper reports both sides of the tradeoff.

Empirical results. I compare three modes on Zipf parametric workloads ($k = 50$, 30 trials per cell):

α	Mode	ε used	Exact hits	Sem. hits	Mean abs. err
0.0	NAIVE	10.0	0.0	0.0	5154.8 (budget exh.)
	WORK	7.0	43.0	0.0	1.0
	SEM L2	1.0	7.3	41.7	5190.1
1.0	NAIVE	10.0	0.0	0.0	7524.4
	WORK	6.9	43.1	0.0	0.8
	SEM L2	1.0	12.2	36.8	3930.5
2.0	NAIVE	10.0	0.0	0.0	8967.1
	WORK	5.6	44.4	0.0	1.0
	SEM L2	1.0	23.2	25.8	1821.9

Interpretation. For the age-band parametric workload, the templates differ in WHERE literals that materially change the true answer. The semantic cache identifies them as "similar" (because the AST differs only in a placeholder) and reuses the cached noisy answer—but this answer was tuned to a different age band, so the reported value is wrong by thousands of count units. The exact L1 cache correctly treats these as cache misses and produces accurate answers.

The dual failure: missing true equivalences too. The dual failure mode is false negatives: query pairs that *are* provably alpha-equivalent should match and save budget. I tested three classical equivalence classes—commutative AND reordering, integer comparator equivalence (`age >= 30` vs `age > 29`), and redundant predicates (`p` vs `p AND 1=1`). Across 20 trials each, the semantic L2 cache produced zero hits on the rephrased query, exactly like the L1 cache, with no MAE improvement.

Two reasons: (i) my Tree Kernel uses ordered children, so commutative reordering produces a strictly different subtree count and the normalized score falls below the 0.95 admission threshold; (ii) the AST embedding (`all-MiniLM-L6-v2`) encodes the canonicalized AST as text, so tokenization-level differences in the reorder push cosine similarity below 0.98.

Conclusion (for this section). The semantic L2 cache fails in both directions: it admits matches on queries with different true answers (false positives), and it misses matches on queries that are provably equivalent (false negatives). Tree-kernel and embedding-based semantic matching are not a free DP optimization. The right architecture is to pair similarity with symbolic equivalence checking—predicate normalization, range merging, alpha-conversion—so the system admits a semantic match only when the queries are provably equivalent, preserving the post-processing property. This is documented as future work in §11.

9 Privacy Leakage Analysis

I complement the budget/utility analysis with two leakage experiments: membership inference and reconstruction. Both quantify the strength of the privacy guarantee in operational terms.

Membership inference. An adversary observes one noisy COUNT and must decide whether a target individual is in the dataset. For $n_{\text{with}} = 500$, $n_{\text{without}} = 499$, sensitivity 1, and the optimal threshold classifier:

ε	MIA AUC (emp.)	MIA acc. (emp.)	$e^{\varepsilon}/(1+e^{\varepsilon})$ (theory)
0.01	0.499	0.496	0.503
0.10	0.522	0.517	0.525
1.00	0.724	0.696	0.731
5.00	0.988	0.959	0.993

Empirical AUC stays below and tracks the DP upper bound $e^{\varepsilon}/(1+e^{\varepsilon})$ within $\leq 1\%$; this expression upper-bounds the optimal attacker's accuracy (the exact single-release optimum is $1 - \frac{1}{2}e^{-\varepsilon/2}$, which the empirical accuracy column matches). At $\varepsilon \leq 0.1$ the attack is no better than chance.

Reconstruction. I replay the milestone's HIV-style differencing attack on a 5-level drill-down over the real Adult table, with nested predicates (all rows; age $\geq$ 30; + sex=Male; + tertiary education; + income>50K) giving true counts (48842, 34327, 24137, 7210, 3137) queried directly from the database. The adversary recovers the per-level differences from noisy releases.

ε	Mean abs. err	p95	Theory $1.5/\varepsilon$
0.01	148.2	404.4	150.0
0.10	14.8	40.4	15.0
1.00	1.5	4.0	1.5
5.00	0.3	0.8	0.3

Mean absolute errors scale as $3/(2\varepsilon) = 1.5/\varepsilon$, the expected absolute value of the difference of two independent $\text{Lap}(1/\varepsilon)$ variables (its standard deviation is the larger $2/\varepsilon$). The empirical mean tracks the $1.5/\varepsilon$ reference to within a few percent across three orders of magnitude in ε. The table is indexed by the *per-release* ε, but the five-level drill-down is five releases: by sequential composition the analyst pays a *cumulative* 5ε in total, so a fixed total budget B split across the levels gives per-release $\varepsilon = B/5$ and correspondingly larger error—the Dinur–Nissim view ties reconstruction to the *total* budget spent, not the per-query value.

Implication for the model. Both attacks degrade in the same regime in which the model predicts more efficient budget consumption (high skew, low ε_q). When the workload is repetitive and the per-query budget can be set tight, the analyst gets both better

Figure 6: Shadow-model MIA AUC across W1–W4 at $\varepsilon_q \in \{0.1, 0.5, 1.0, 2.0\}$. Even with 32 cells ×60 shadow runs and a logistic-regression classifier observing the full $k = 50$ query sequence, empirical AUC stays well below the cumulative-budget upper bound (which approaches 1.0). Most queries do not directly probe the target tuple.

utility and stronger privacy. When the workload is a drill-down (W4), the model correctly predicts that workload-aware caching does *not* help and the reconstruction attack accordingly succeeds at any practical ε.

Workload-level shadow-model MIA (across W1–W4). The single-query MIA above is conservative: in practice, an attacker sees a *sequence* of releases and may train a classifier (a "shadow model") on simulated runs of the middleware. I executed this stronger attack to address the reviewer's specific request for MIA "across W1–W4 at multiple ε values."

Setup. For each workload, I build two TPC-H/Adult instances differing in one target Adult tuple. Each instance produces an i.i.d. noisy output sequence under the middleware. I collect 30 shadow runs per world (60 sequences total), train a logistic-regression classifier on 70% of them, and measure AUC on the holdout 30%.

Result. Across 4 workloads × 4 epsilons × 2 modes (32 cells), the empirical shadow MIA AUC ranges from 0.14 to 0.58. The meaningful reference is the random-guess line 0.5: most cells sit at or below it, and even the strongest (W1 repetitive, $\varepsilon_q = 2$) reaches only 0.58. The *cumulative* bound $e^{\sum \varepsilon_q}/(1+e^{\sum \varepsilon_q})$ saturates at ≈ 1.0 once total consumption exceeds ~ 5, so comparing against it is vacuous; the informative comparison is the per-query bound $e^{\varepsilon_q}/(1+e^{\varepsilon_q})$, which the empirical AUC also stays under. *Caveat:* with 60 sequences per cell and an 18-sequence holdout, cells reporting AUC < 0.5 reflect small-sample estimator variance (an anti-correlated classifier on a tiny test set), not sub-chance leakage—the honest reading is that workload-level leakage is *weak*, not that it is below chance.

Why the gap? Most queries in W1–W3 do not directly probe the target row; they are diluted across the $k = 50$-query workload. The classifier learns to identify the few informative queries, but the signal-to-noise ratio is governed by the per-query ε_q, not the cumulative budget. This empirical finding is consistent with prior work on amplification by sub-sampling and demonstrates that *cumulative-budget bounds are loose for realistic workload MIA.*

Cross-check with the R2 model. The R2 model predicts an upper bound on attacker advantage via the cumulative ε consumed. The empirical AUC falls well below this bound in all 32 cells, confirming that the model is conservative for leakage prediction (it overestimates risk, which is the safe direction). A tighter joint model of budget → attack success is left as future work.

10 Predictive Budget Allocator: Putting the Model to Work

The analytical model of §4 is descriptive: given a workload distribution, it predicts how much budget will be consumed. In this section I close the loop and use the same model *prescriptively*, as the backbone of a new execution mode that adapts the per-query privacy parameter ε_q online instead of fixing it offline.

Mechanism. At each incoming query, the allocator maintains an empirical frequency distribution over observed template hashes (Laplace-smoothed). After a small warmup phase (3–10% of the workload, used to seed the prior), it estimates the workload's total unique-template count as $\hat{U} = \mathbb{E}[u_{k_{\text{total}}}]$ under $\hat{p}$ from Proposition 1, and allocates

$$\varepsilon_q^{\text{pred}} = \frac{B}{\max(\hat{U}, 1)},$$

where B is the total privacy budget. The allocator caps the per-query value to the remaining budget so total spend never exceeds B by construction.

Algorithm 2 makes this decision explicit. Unlike Algorithm 1, whose per-query budget is a fixed input, here ε_q is a *runtime function* of the forecasted unique-template count $\hat{U}$: the non-trivial logic is the online re-estimation of $\hat{U}$ from the empirical template distribution and the resulting $B/\hat{U}$ allocation.

Algorithm 2 Predictive per-query budget allocation

Require: Query q, total budget B, declared length k_{tot}, warmup size w, floor ε_0, consumed c
Ensure: Per-query budget ε_q (0 = serve from cache or reject)
1: Append q's template hash to history; update the unique-template set
2: **if** $B - c \le \varepsilon_0$ **then**
3: **return** 0 (budget exhausted; exact-repeat cache hits still served free)
4: **end if**
5: **if** queries seen $\le w$ **then**
6: **return** $\max(\varepsilon_0,\ \min(B/k_{\text{tot}},\ B - c))$ (warmup: per-slot average under a flat prior)
7: **end if**
8: $\hat{p} \leftarrow$ Laplace-smoothed empirical template frequencies from history
9: $\hat{U} \leftarrow \mathbb{E}[u_{k_{\text{tot}}}]$ under $\hat{p}$ (Proposition 1)
10: **return** $\max(\varepsilon_0,\ \min(B/\max(\hat{U}, 1),\ B - c))$ (offline-optimal target $\varepsilon_q^* = B/u_k$)

Why this is novel. PINQ [18], PrivateSQL [17], Chorus [16], and DOP-SQL [24] all expose a fixed per-query ε_q that the analyst chooses up front. None of them set ε_q from a forecast of *future workload-aggregate* consumption. Three systems adapt budget in adjacent senses: APEx [32] translates a per-query accuracy bound

into the minimum-privacy ε for the *current* query at runtime, PrivateSQL [17] allocates budget across views *offline* over a representative workload, and BGTplanner [14] learns a predict-then-bandit budget policy for federated-learning rounds (a Gaussian process plus a contextual bandit), not a closed-form forecast for a query workload. The distinction here is that the predictive allocator chooses ε_q from an analytical forecast of future unique-template consumption (Proposition 1), rather than from the current query's accuracy target or a static per-view split. Its mathematical justification rests directly on Proposition 2: the optimal offline allocation is $\varepsilon_q^* = B/u_k$, and the predictive allocator approximates u_k online.

Empirical comparison. Table 6 compares three strategies on Zipf workloads ($k = 100$, $B = 10$, 30 trials per cell). Naive divides B evenly across k queries ($\varepsilon_q = 0.1$). Workload-aware uses the same fixed $\varepsilon_q = 0.1$ but only charges cache misses—it answers all 100 queries but never spends more than $\sim 0.7\varepsilon$ of the budget. Predictive uses the model to choose ε_q on the fly.

Interpretation. The predictive allocator delivers a modest MAE reduction (point estimates 16.8% at $\alpha = 0$ down to 5.4% at $\alpha = 2$) at the cost of denying some queries when the budget runs out. We are deliberately careful with this number: a paired t-test over the 30 trials per cell finds the gain significant only at low skew ($p \approx 0.05$ at $\alpha = 0$, $p > 0.3$ for $\alpha \geq 1$, with the allocator winning in only 18/30 trials at $\alpha = 2$), so it is best read as a low-skew upper bound rather than a uniform improvement. The robust, regime-independent allocation result is instead the safe $\varepsilon_q = B/m$ policy (Table 8), whose margin over a learned bandit is $\gtrsim 9\times$ its standard error (gap 1.8 vs. per-policy SE 0.06–0.19). The mechanism is visible in the per-release budget, and we describe it precisely: because the parametric age-band pool collapses to a *single* structural template once `WHERE` literals are placeholdered ($\hat{U} \approx 1$), the allocator spends the warmup at $\varepsilon_q = 0.1$ and then concentrates the residual budget on the first distinct miss (a measured mean of ≈2–3 per miss), rather than the uniform B/m that a genuine m-template pool would give—a degenerate but faithful instance of $\varepsilon_q = B/\hat{U}$. Cache hits inherit whichever noisy answer was released at the time, so the cache becomes "locked in" to the early-release quality. To realize the full B/U utility benefit, the mechanism must *re-release* cached answers when the budget shows surplus—an extension I discuss in §11.

The gap across budget regimes. The MAE improvement at $B = 10$ is not a one-off. Sweeping the total budget over $B \in \{1, 5, 10, 50\}$ at $\alpha = 1$, $k = 100$ (20 trials per cell) shows MAE reductions of 32% (B=1: predictive 68.7 vs naive 101.1), ~0.5% (B=5, where naive and predictive are near-identical), 18% (B=10), and 21% (B=50: 1.58 vs 2.0) relative to naive. The reduction is largest in the tightest-budget regime, where every release matters.

Scale invariance. The allocator's advantage persists at TPC-H SF=10 (60M lineitem rows, 20 trials, $\alpha = 1$, $k = 100$, $B = 10$): predictive cuts MAE to 6.03 vs naive's 10.01, a 40% reduction at 10× the data volume. This is consistent with the model being structural, not data-dependent.

Composing with the temporal regime. The two new mechanisms compose. Combining the predictive allocator with a finite staleness tolerance τ (Section 5) yields lower MAE than temporal alone at every τ tested (e.g. MAE 1.25 vs 1.76 at $\tau = 100$, $\alpha = 1$, $B = 50$, 20 trials), in exchange for spending the full budget rather than the temporal-alone fraction. The choice is a deployment preference: cheap re-noising with higher noise (temporal alone) versus full-budget spend with the lowest achievable noise (combined).

Which estimator predicts u_k? The allocator's quality hinges on how well it predicts u_k from the warmup prefix. The default plug-in occupancy estimate (§10) is the *naive* unseen-species estimator: its support is only the observed templates, so it cannot count the unseen and systematically under-predicts. Table 7 compares it against the horizon-aware alternatives (Section 2) on Zipf workloads ($m = 50$ templates, $k = 100$, 40 trials), as a function of the extrapolation factor $t = (k - n)/n$ (so $t = 1$ means the warmup is half the workload). In the realistic regime where the warmup is at least a third of the workload ($t \leq 2$), the Good–Toulmin family roughly halves the prediction error and is near-unbiased, while the plug-in under-predicts by 18–43%; raw Good–Toulmin diverges past its $t \leq 1$ validity limit, and the smoothed estimator [11] tames it (27% vs 193% at $t = 2$). For very short warmups ($t = 4$, warmup $< k/4$) no extrapolator is reliable—beyond the $t \propto \log n$ range—so a conservative bound is the safe choice. Chao1 is roughly flat and increasingly over-predicts, confirming that it answers total richness, not the horizon-k count. The smoothed estimator improves the *prediction* accuracy, so we make it a selectable mode of the live allocator (`estimator='smoothed_gt'` in `src/dpdb/predictive.py`); the estimators themselves live in `src/dpdb/predictors.py`. Its effect on the *allocation*, however, is a trade-off rather than a free win, and we report it as such: on the Zipf allocation benchmark (Table 8) switching the allocator from plug-in to smoothed-GT lifts the answered fraction from 81% to 92%—its over-prediction of $\hat{U}$ makes the allocator spend more conservatively, so fewer late queries are rejected—but at higher per-release noise. Better u_k accuracy is thus not automatically better allocation: smoothed-GT is the right default when coverage matters, the plug-in when per-release noise does.

Closed-form vs. a learned (bandit) allocation policy. The estimator above answers *how to predict u_k*; a separate question is *which policy* should spend the prediction. BGTplanner [14] (Section 2) allocates budget online with a contextual bandit driven by *observed utility*—validation accuracy in federated learning. In DP-SQL the per-query error is not observable (the true answer is exactly what privacy hides), so a utility-driven bandit cannot see its reward; we give it instead an ε-greedy policy over denominators $d \in \{k, k/2, k/4, m\}$ ($\varepsilon_q = B/d$), rewarded by the granted ε_q and penalised on a budget-exhausted rejection—learning from the only *observable* signals. We benchmark it against closed-form allocators on Zipf workloads ($m = 20$, $k = 100$, $B = 10$, 60 trials; harness `experiments/allocation_policy_comparison.py`): naive ($\varepsilon_q = B/k$); the *safe* closed form $\varepsilon_q = B/m$; and an oracle $\varepsilon_q = B/u_k$ that knows the realised distinct count. The safe point uses the one hard bound available offline—there are only m templates, so $u_k \leq m$ and $(B/m)\, u_k \leq B$, hence it *never* rejects and needs no warmup. We report *fresh-release* MAE (over cache misses, isolating allocation quality) jointly with the fraction answered, since noise and coverage trade off and either figure alone misleads.

Table 6: Predictive allocator vs fixed-ε_q baselines (mean±SE over 30 trials, $k = 100$, total budget $B = 10$, warmup fraction 3%). The predictive MAE gain over naive is significant only at $\alpha = 0$ (paired t-test, $p \approx 0.05$); at $\alpha \geq 1$ it is within sampling noise ($p > 0.3$, allocator winning 18–22/30 trials). WORKLOAD-DP can be marginally *worse* than naive because it reuses a single noise draw rather than averaging independent ones—the budget/variance trade-off.

α	Mode	ε spent	MAE	Queries answered	ε_q per release
0.0	NAIVE	10.0	9.86	100/100	0.10
	WORKLOAD-DP	0.7	9.46	100/100	0.10 (per miss)
	PREDICTIVE	10.0	**8.21** ± 0.85	69/100	$B/\hat{U}$, $\hat{U}$=1
1.0	NAIVE	10.0	9.81	100/100	0.10
	WORKLOAD-DP	0.7	9.96	100/100	0.10 (per miss)
	PREDICTIVE	10.0	**8.98** ± 0.93	78/100	$B/\hat{U}$, $\hat{U}$=1
2.0	NAIVE	10.0	9.96	100/100	0.10
	WORKLOAD-DP	0.7	10.22	100/100	0.10 (per miss)
	PREDICTIVE	10.0	**9.42** ± 1.40	88/100	$B/\hat{U}$, $\hat{U}$=1

Table 7: Mean absolute error of the u_k prediction (%), by estimator and extrapolation factor $t = (k - n)/n$ (Zipf workloads, $m = 50$, $k = 100$, averaged over $\alpha \in \{0.5, 1, 1.5, 2\}$, 40 trials). Lower is better; the plug-in under-predicts at every t.

u_k estimator	t=0.5	t=1	t=2	t=4
Plug-in occupancy (current)	18	30	43	58
Good–Toulmin [8]	**8**	**17**	193	204
Smoothed GT [11]	**8**	**17**	**27**	151
Chao1 richness (ref.) [10]	55	48	47	47

Result: the closed form reaches the bandit's best case at zero exploration cost. Table 8 is layered, and we state each layer precisely. Naive wastes the budget: $\varepsilon_q = B/k$ assumes k distinct queries but only $u_k \approx 16$ occur, so it spends 1.6 of $B = 10$ and leaves 84% unused (MAE 10). The safe closed form answers *every* query at fresh MAE 2.0, against the bandit's 3.8 at the same 100% rate—a 1.9× reduction (gap 1.8 vs. per-policy SE 0.06–0.19, i.e. $\gtrsim 9\times$ the standard error, stable across seeds, independently reproduced). The mechanism is not a noise-scale advantage: $\varepsilon_q = B/m$ *is* the operating point of the bandit's best arm ($d = m$), so the closed form reaches the bandit's best case *directly* while the bandit pays an exploration tax sampling its three worse arms. A bandit constrained to answer 100% can only *tie* the safe point—equivalently, a trivial greedy that always plays the largest-ε arm (d=m) *is* the safe allocator, so the 1.9× gap is the bandit's exploration tax, not an advantage of the learned policy class over a one-line rule. This favourable comparison rests on the bandit's finest denominator being d=m, i.e. $m \leq k/4$; we surface that condition rather than bury it. The single way a bandit prints a lower fresh MAE is to spend below B/m ($d < m$), which starts *rejecting* late distinct templates (8–23% dropped for MAE 1.3–1.5)—rejection-bought, which the joint table exposes. Going below B/m *safely* instead requires knowing u_k in advance: the oracle reaches MAE 1.6 at full coverage. So B/m is the best any *model-free* policy can do at 100% answered, and *only a forecast of* u_k (Section 10) crosses that floor without starving later queries—the learned bandit, lacking one, cannot. This is the paper's thesis as an allocation result: predicting u_k is the lever; the allocator is one consumer of it. Two caveats bound the claim. The 1.9× figure holds when $m \leq k/4$ (so the bandit's finest denominator is $d = m$); for $m > k/4$ its grid contains a $d < m$ that reaches a finer scale—though still by under-answering. And $\varepsilon_q = B/m$ is itself conservative (m over-estimates the realised u_k, underspending to 7.9 of B): a safe floor, not the optimum. DP validity rests on m and u_k being public properties of the query stream, not of the protected rows—verified, as every policy's ε choices are bit-identical under noise-RNG variation.

Table 8: Online allocation policies (Zipf, $m = 20$, $k = 100$, $B = 10$, averaged over $\alpha \in \{0.5, 1, 1.5, 2\}$, 60 trials). *Fresh MAE* is over cache misses (allocation quality), reported jointly with the fraction answered. The safe closed form ε_q=B/m answers all queries at the bandit's best-case noise scale with no exploration tax; only a u_k forecast (oracle) safely goes lower.

Policy	Fresh MAE	Answered	ε used
Naive (ε_q=B/k)	10.1	100/100	1.6
ε-greedy bandit	3.8	100/100	6.1
Closed form, safe (ε_q=B/m, ours)	**2.0**	**100/100**	8.0
Oracle (ε_q=B/u_k, needs forecast)	1.6	100/100	10.0
Bandit forced below B/m ($d < m$ arms): fresh MAE 1.3–1.5 but only 77–92/100 answered—rejection-bought.			

11 Future Work

The analytical model and the predictive allocator open several concrete next steps, all of which were considered out of scope for the course-project version of this work and are deferred here.

Cache re-release on budget surplus. The predictive allocator's main weakness is that early-release cache entries lock in the noise scale used at the time. If the workload turns out more skewed than initially predicted, late-stage budget surplus cannot be used to refresh those entries. A re-release policy that periodically reruns the noise mechanism on stale cache entries when surplus exceeds a threshold would close this gap. The privacy accounting is straightforward (each re-release is a fresh ε_q-DP release).

Advanced composition with Rényi DP.. My budget ledger uses basic sequential composition: $\varepsilon_{\text{total}} = \sum \varepsilon_i$. Switching to Rényi DP [19]

or zCDP [42] would tighten the bound on the total privacy loss, allowing more releases (or larger per-query ε_q) under the same final (ε, δ) guarantee. The R2 model would then predict $\sum \varepsilon_i$ rather than count u_k, with the closed-form structure mostly unchanged.

Burstiness and renewal-process arrival models. The i.i.d. template sampling assumption in §4 is a useful first approximation. Real workloads exhibit bursts: a dashboard refreshes 30 times in a minute, then sleeps for an hour. Replacing the Poisson assumption with a renewal process (Pareto or hyperexponential inter-arrival distribution) would let the temporal extension capture these patterns while preserving the closed-form $\mathbb{E}[\varepsilon]$ derivation.

Real-world workload traces. The Zipf parametrization in this paper is a controlled sweep, not a measurement of real analyst behaviour. Replicating the experiments against a production-style query log (e.g., the Snowflake or Redshift trace anonymizations published in recent benchmark papers) would validate the model under genuinely heavy-tailed real workloads.

Cross-checking the model with leakage bounds. §9 showed that the cumulative-ε upper bound is loose for shadow-model MIA on realistic workloads. A tighter joint bound that uses the workload's $\mathbb{E}[u_k]$ to discount the per-target privacy loss is a clear next theoretical step.

Equivalence-checked semantic cache. The semantic L2 cache of §8 fails because tree-kernel and embedding similarity do not imply alpha-equivalence. Pairing those similarity tools with a symbolic equivalence prover (predicate normalization, range-merging, etc.) would let the system safely admit a semantic match only when the queries are provably equivalent, preserving the post-processing property.

Private group-key domains. The GROUP BY accounting (Section 3) charges only the aggregate values and assumes the set of group keys is a public categorical domain fixed by the schema. For sensitive or unbounded key domains (free-text, high-cardinality identifiers), the present mechanism would reveal membership through a singleton group; closing this requires a stability-based release that adds noise to each group count and *suppresses* groups whose noisy count falls below a threshold (the standard DP-histogram construction), at a small δ cost. Integrating such thresholding—and extending the model to predict the budget of the surviving groups—is a clean next step.

Multi-table and JOIN support. The current scope (single-table aggregates) was deliberately approved at the milestone stage. The model's framework—occupancy over a template distribution—generalizes naturally to JOIN-shaped queries if a sensitivity bound (residual sensitivity [43] or R2T [44]) is available per template. Incorporating join sensitivity is the most direct path to a thesis-grade extension.

Decomposed AVG with per-group accounting. The current implementation collapses AVG to a single Laplace release with sensitivity B_c (worst case: group of size one). A SUM+COUNT decomposition would charge $2\varepsilon_q$ per group but apply noise of scale B_c/ε_q to the SUM and $1/\varepsilon_q$ to the COUNT, dividing through to yield mean-error roughly $O(B_c/(n\varepsilon_q))$ for groups of size n. For TPC-H scales where most groups have $n \gg 1$ (e.g., `l_returnflag` groups in `lineitem`), this is a measurable improvement. The change touches only the analyzer and mechanism layer; the budget ledger and predictive allocator already support multi-aggregate sequential composition.

12 Discussion and Conclusion

What I found. Four findings stand out.

(1) **Budget consumption is a function of $\mathbb{E}[u_k]$, not the cache.** $\mathbb{E}[\varepsilon_{\text{wa}}] = \varepsilon_q \mathbb{E}[u_k]$ holds within $< 3\%$ in 21 of 24 main-grid cells, *exactly* on the constructed mixed workload, and is invariant to scale (SF=1 vs SF=10) and to ε_q—so a deployment can estimate its privacy cost *before* running any query.
(2) **The $1 - 1/k$ rule is a corner case.** It is Limit A ($\alpha \to \infty$) of the model, which also predicts the full interior up to naive composition (96.5% predicted and empirical at $\alpha = 2$, $k = 200$).
(3) **The cumulative-ε bound is loose for realistic MIA.** Shadow-model AUC across W1–W4 stays at 0.14–0.58, far below the bound's $\to$ 1.0; worst-case accounting overspends budget.
(4) **Semantic similarity is not equivalence.** Tree-kernel + embedding matching admits false positives (different answers, similar ASTs) and misses textbook equivalences, so a "semantic DP cache" is unsafe without a symbolic equivalence prover.

The model also flags its failure regimes—uniform workloads and drill-downs ($\alpha \to 0$, W4), where every query is unique, savings vanish, and reconstruction succeeds at any practical ε—and makes the cost of freshness explicit (staleness τ=10 over horizon T=100 costs ~10× the static budget, as an upper bound).

Threats to validity. I separate three. *Construct.* The headline "$\mathbb{E}[u_k]$ within $< 3\%$ in 21/24 cells" (Section 7) is a Monte-Carlo *self-consistency* check, not an external-validity test: the empirical u_k is sampled from the very Zipf distribution the closed form integrates, so the 30-trial mean converges to $\mathbb{E}[u_k]$ *by construction*. It validates the estimator's arithmetic and convergence, not the claim that real workloads are i.i.d. Zipf. Per-release MAE excludes rejected queries, so we report the answered fraction in the same cells. *Internal.* The budget forecast assumes a *non-adaptive* analyst whose template choices are independent of the released noise. Privacy is unaffected—DP composition is adaptive-safe and total spend is hard-capped at B by construction—but $\mathbb{E}[u_k]$ is an *average-case* forecast: against a worst-case adaptive analyst the only valid bound is $u_k \le \min(k, m)$, i.e. $\varepsilon_{\text{wa}} \le \varepsilon_q m$, which is exactly the B/m safe allocator (Table 8). The constructed mixed-workload agreement is a ledger identity (u_k is fixed by construction, so $\varepsilon_q u_k$ is definitional with zero trial variance), not an independent prediction. *External.* All workloads are synthetic Zipf sweeps plus TPC-H/Adult parametric families; with no real analyst query trace, generalization to production heavy-tailed, bursty, or correlated workloads is untested—the single largest threat. Scope is single-table aggregates under basic sequential composition; JOINs, tight (Rényi/zCDP/PLD) composition, and renewal-process arrivals are out of scope (Section 11).

Reproducibility. The middleware, all experiment generators, and the analysis scripts are in the repository, and the canonical result CSVs the tables cite are committed under `results/`. The model-validation, predictor, and allocation experiments use fixed integer seeds and are bit-reproducible (the analysis of the predictive table is regenerated by `experiments/analyze_predictive.py`); the large six-sweep campaign is *distributionally* reproducible—re-running its generators yields statistically equivalent numbers, and the committed CSVs from the submitted runs are the canonical artifact.

References

[1] D. E. Denning. 1979. The Tracker: A Threat to Statistical Database Security. *ACM Transactions on Database Systems* 4, 1, 76–96.

[2] I. Dinur and K. Nissim. 2003. Revealing Information while Preserving Privacy. In *PODS*, 202–210.

[3] C. Dwork, F. McSherry, K. Nissim, and A. Smith. 2006. Calibrating Noise to Sensitivity in Private Data Analysis. In *TCC*, 265–284.

[4] C. Dwork, G. N. Rothblum, and S. Vadhan. 2010. Boosting and Differential Privacy. In *FOCS*, 51–60.

[5] C. Dwork and A. Roth. 2014. *The Algorithmic Foundations of Differential Privacy.* F&T in Theor. CS 9, 3–4, 211–407.

[6] P. Flajolet, D. Gardy, and L. Thimonier. 1992. Birthday Paradox, Coupon Collectors, Caching Algorithms and Self-Organizing Search. *Discrete Applied Mathematics* 39, 3, 207–229.

[7] I. J. Good. 1953. The Population Frequencies of Species and the Estimation of Population Parameters. *Biometrika* 40, 3–4, 237–264.

[8] I. J. Good and G. H. Toulmin. 1956. The Number of New Species, and the Increase in Population Coverage, When a Sample is Increased. *Biometrika* 43, 1–2, 45–63.

[9] B. Efron and R. Thisted. 1976. Estimating the Number of Unseen Species: How Many Words Did Shakespeare Know? *Biometrika* 63, 3, 435–447.

[10] A. Chao. 1984. Nonparametric Estimation of the Number of Classes in a Population. *Scandinavian Journal of Statistics* 11, 4, 265–270.

[11] A. Orlitsky, A. T. Suresh, and Y. Wu. 2016. Optimal Prediction of the Number of Unseen Species. *PNAS* 113, 47, 13283–13288.

[12] P. Flajolet, É. Fusy, O. Gandouet, and F. Meunier. 2007. HyperLogLog: the Analysis of a Near-optimal Cardinality Estimation Algorithm. In *AofA*.

[13] L. Ma, D. Van Aken, A. Hefny, G. Mezerhane, A. Pavlo, and G. J. Gordon. 2018. Query-based Workload Forecasting for Self-Driving Database Management Systems. In *SIGMOD*, 631–645.

[14] X. Zhang, Y. Zhou, M. Hu, D. Wu, P. Liao, M. Guizani, and M. Sheng. 2024. BGTplanner: Maximizing Training Accuracy for Differentially Private Federated Recommenders via Strategic Privacy Budget Allocation. arXiv:2412.02934 (IEEE Trans. Services Computing, 2025).

[15] N. Johnson, J. P. Near, and D. Song. 2018. Towards Practical Differential Privacy for SQL Queries. *PVLDB* 11, 5, 526–539.

[16] N. Johnson, J. P. Near, J. Hellerstein, and D. Song. 2020. Chorus: A Programming Framework for Building Scalable Differential Privacy Mechanisms. In *EuroS&P*.

[17] I. Kotsogiannis, Y. Tao, X. He, M. Fanaeepour, A. Machanavajjhala, M. Hay, and G. Miklau. 2019. PrivateSQL: A Differentially Private SQL Query Engine. *PVLDB* 12, 11, 1371–1384.

[18] F. McSherry. 2009. Privacy Integrated Queries: An Extensible Platform for Privacy-preserving Data Analysis. In *SIGMOD*, 19–30.

[19] I. Mironov. 2017. Rényi Differential Privacy. In *CSF*, 263–275.

[20] M. Abadi, A. Chu, I. Goodfellow, H. B. McMahan, I. Mironov, K. Talwar, and L. Zhang. 2016. Deep Learning with Differential Privacy. In *ACM CCS*, 308–318.

[21] S. Gopi, Y. T. Lee, and L. Wutschitz. 2021. Numerical Composition of Differential Privacy. In *NeurIPS*.

[22] Z. Yang, E. Liang, A. Kamsetty, C. Wu, Y. Duan, X. Chen, P. Abbeel, J. M. Hellerstein, S. Krishnan, and I. Stoica. 2019. Deep Unsupervised Cardinality Estimation. *PVLDB* 13, 3, 279–292.

[23] NIST. 2025. Guidelines for Evaluating Differential Privacy Guarantees. *NIST SP 800-226*.

[24] J. Yu, W. Dong, J. Fang, D. Sun, and K. Yi. 2024. DOP-SQL: A General-purpose, High-utility, and Extensible Private SQL System. *PVLDB* 17, 12, 4385–4388.

[25] W. Dong, D. Sun, and K. Yi. 2023. Better than Composition: How to Answer Multiple Relational Queries under DP. *PACMMOD* 1, 2.

[26] M. Collins and N. Duffy. 2001. Convolution Kernels for Natural Language. In *NeurIPS*, 625–632.

[27] C. Li, M. Hay, V. Rastogi, G. Miklau, and A. McGregor. 2010. Optimizing Linear Counting Queries Under Differential Privacy. In *PODS*, 123–134.

[28] R. McKenna, G. Miklau, M. Hay, and A. Machanavajjhala. 2018. Optimizing Error of High-Dimensional Statistical Queries Under Differential Privacy. *PVLDB* 11, 10, 1206–1219.

[29] M. Hardt, K. Ligett, and F. McSherry. 2012. A Simple and Practical Algorithm for Differentially Private Data Release. In *NeurIPS*, 2339–2347.

[30] A. Roth and T. Roughgarden. 2010. Interactive Privacy via the Median Mechanism. In *STOC*.

[31] M. Hardt and G. N. Rothblum. 2010. A Multiplicative Weights Mechanism for Privacy-Preserving Data Analysis. In *FOCS*, 61–70.

[32] C. Ge, X. He, I. F. Ilyas, and A. Machanavajjhala. 2019. APEx: Accuracy-Aware Differentially Private Data Exploration. In *SIGMOD*.

[33] I. Kotsogiannis, A. Machanavajjhala, M. Hay, and G. Miklau. 2017. Pythia: Data Dependent Differentially Private Algorithm Selection. In *SIGMOD*.

[34] S. Berghel et al. 2022. Tumult Analytics: a robust, easy-to-use, scalable, and expressive framework for differential privacy. arXiv:2212.04133.

[35] OpenDP Project and SmartNoise. 2023. OpenDP Library and SmartNoise SQL. https://opendp.org/.

[36] R. J. Wilson, C. Y. Zhang, W. Lam, D. Desfontaines, D. Simmons-Marengo, and B. Gipson. 2020. Differentially Private SQL with Bounded User Contribution. *PoPETs* 2020, 2, 230–250.

[37] S. Zhang and X. He. 2023. DProvDB: Differentially Private Query Processing with Multi-Analyst Provenance. *Proc. ACM Manag. Data* 1, 4, Article 267 (presented at SIGMOD 2024). arXiv:2309.10240.

[38] B. Zhang, V. Doroshenko, P. Kairouz, T. Steinke, A. Thakurta, Z. Ma, E. Cohen, H. Apte, and J. Spacek. 2024. Differentially Private Stream Processing at Scale (DP-SQLP). *PVLDB* 17, 12. arXiv:2303.18086.

[39] N. Grislain, P. Roussel, and V. de Sainte Agathe (Sarus Technologies). 2024. Qrlew: Rewriting SQL into Differentially Private SQL. arXiv:2401.06273. Open-source: https://github.com/Qrlew/qrlew.

[40] T. Matsumoto, S. Takakura, S. Takagi, and S. Hasegawa. 2026. DPSQL+: A Differentially Private SQL Library with a Minimum Frequency Rule. arXiv:2602.22699.

[41] M. Hay, A. Machanavajjhala, G. Miklau, Y. Chen, and D. Zhang. 2016. Principled Evaluation of Differentially Private Algorithms using DPBench. In *SIGMOD*.

[42] M. Bun and T. Steinke. 2016. Concentrated Differential Privacy: Simplifications, Extensions, and Lower Bounds. In *TCC*, 635–658.

[43] W. Dong and K. Yi. 2021. Residual Sensitivity for Differentially Private Multi-Way Joins. In *SIGMOD*, 432–445.

[44] W. Dong, J. Fang, K. Yi, Y. Tao, and A. Machanavajjhala. 2022. R2T: Instance-optimal Truncation for Differentially Private Query Evaluation with Foreign Keys. In *SIGMOD*, 759–772.

[45] K. Kostopoulou, P. Tholoniat, A. Cidon, R. Geambasu, and M. Lécuyer. 2023. Turbo: Effective Caching in Differentially-Private Databases. In *SOSP*. arXiv:2306.16163.

[46] M. Mazmudar, T. Humphries, J. Liu, M. Rafuse, and X. He. 2023. Cache Me If You Can: Accuracy-Aware Inference Engine for Differentially Private Data Exploration. *PVLDB* 16, 4. arXiv:2211.15732.

[47] C. Dwork, M. Naor, T. Pitassi, and G. N. Rothblum. 2010. Differential Privacy Under Continual Observation. In *STOC*, 715–724.

A Glossary of Key Concepts

This appendix collects the terminology used in the paper for readers new to DP-SQL budgeting, grouped by theme; each entry is a one-line definition.

Privacy primitives.

Differential privacy (DP) adding calibrated noise so a single record's presence is statistically hidden.

Privacy budget ε the DP knob: small ε = more private, noisier; large ε = accurate, less private. Each release spends from a finite total B.

Laplace mechanism the noise we add, at scale $\Delta f/\varepsilon_q$.

Sensitivity Δf how much one record can change an answer (COUNT: 1).

Sequential composition the budget ledger: $\varepsilon_{\text{total}} = \sum_i \varepsilon_i$.

Post-processing reusing an already-noised answer is free (no budget); the basis of caching.

Threat model unbounded, tuple-level (add/remove one row); the query stream is the analyst's, not protected data.

Workload model.

Template a query's structural shape; identical repeats are the same template.

u_k the number of *distinct* templates in k queries — the quantity the whole model predicts.

Occupancy model the closed form $\mathbb{E}[u_k] = \sum_i [1-(1-p_i)^k]$ (coupon-collector).

Savings $S(k)$ $1 - \mathbb{E}[u_k]/k$: the budget fraction repetition lets us avoid spending.

Predicting u_k (unseen-species estimation).

Plug-in estimator feeding observed frequencies into $\mathbb{E}[u_k]$; under-predicts (its support is only seen templates).

Unseen-species problem estimating not-yet-observed templates (the reason plug-in is biased).

Good–Toulmin / Smoothed GT horizon-k extrapolators that halve the prediction error for extrapolation factor $t \leq 2$.

Chao1 an asymptotic richness estimator (total, not horizon-k); included as a reference.

Extrapolation factor t $(k-n)/n$: how far past the warmup of size n we predict.

Online budget allocation.

Predict-then-allocate our closed-form policy $\varepsilon_q = B/\hat{U}$ (training-free).

BGTplanner / predict-then-bandit the closest prior work: a *learned* GP+bandit budgeter (federated learning).

ε-greedy bandit an online learner that explores allocation choices; our learned baseline.

Fresh-release MAE error over cache misses only, isolating allocation quality; reported with % answered.

Safe operating point $\varepsilon_q = B/m$ since $u_k \leq m$, this never rejects and matches the bandit's best arm with no exploration.

Exploration tax the noise a bandit pays sampling worse arms before finding the operating point a closed form computes directly.

Oracle allocator $\varepsilon_q = B/u_k$ with u_k known: the upper bound that only a forecast can approach.